Smart Chef & Pantry AI

Document 16 — Deployment, Environment & DevOps Specification

1. Dokümanın Amacı

Bu dokümanın amacı Smart Chef & Pantry AI projesinin geliştirme, test ve production ortamlarının teknik olarak tanımlanması ve uygulamanın güvenli, kontrollü ve tekrarlanabilir şekilde çalıştırılmasını sağlamaktır.

Bu dokümanda:

- Development, test ve production ortamları
- Flutter uygulamasının yapılandırılması
- Flask backend deployment yapısı
- PostgreSQL deployment yaklaşımı
- Environment variables
- Secret management
- Database migration
- Build ve release süreci
- CI/CD
- Logging
- Monitoring
- Backup
- Rollback
- HTTPS
- Production security
- Deployment checklist

tanımlanacaktır.

Bu doküman Document 01–15'te tanımlanan mimari ve MVP kapsamıyla uyumludur.

---

2. Deployment Kapsamı

Smart Chef & Pantry AI temel olarak üç ana deployment bileşeninden oluşacaktır:

Flutter Mobile Application
↓
Flask REST API

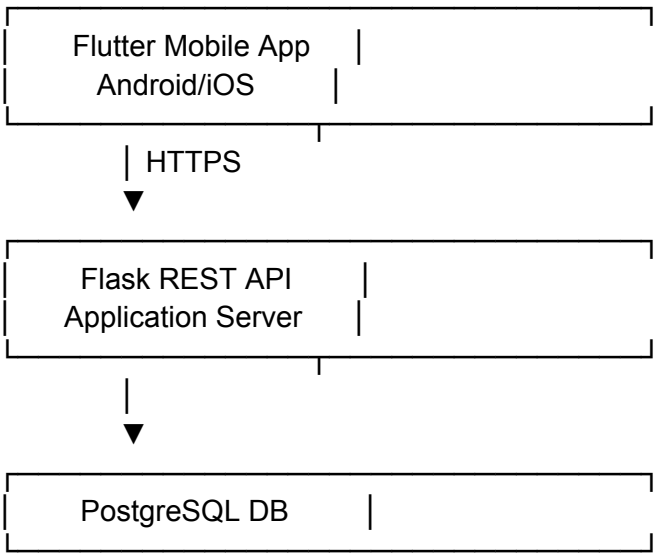
↓
PostgreSQL Database

Ek olarak:

Google Authentication

harici authentication servisi olarak kullanılacaktır.

Genel yapı:



---

3. Environment Yapısı

Projede minimum üç environment bulunmalıdır:

Development
Testing
Production

Bunların birbirinden ayrılması zorunludur.

Development

Geliştiricilerin günlük geliştirme yaptığı ortamdır.

Özellikleri:

- Debug açık olabilir.
- Local PostgreSQL kullanılabilir.
- Test kullanıcıları kullanılabilir.
- Ayrı Google OAuth configuration kullanılabilir.
- Ayrı JWT secret kullanılmalıdır.
- Production verisi kullanılmamalıdır.

Testing

Automated testlerin çalıştırıldığı ortamdır.

Özellikleri:

- Production database kullanılmaz.
- Test database kullanılır.
- Test secrets kullanılır.
- Test data kullanılır.
- CI/CD pipeline tarafından kullanılabilir.

Production

Gerçek kullanıcıların kullandığı ortamdır.

Özellikleri:

- Debug kapalıdır.
- HTTPS zorunludur.
- Production PostgreSQL kullanılır.
- Production secrets kullanılır.
- Detaylı hata bilgileri kullanıcıya gösterilmez.
- Database backup uygulanır.
- Logging ve monitoring aktif olmalıdır.

---

4. Environment Isolation

Ortamlar kesin olarak birbirinden ayrılmalıdır.

Yanlış yapı:

Development
↓
Production Database

Doğru yapı:

Development → Development DB
Testing → Test DB
Production → Production DB

Bir geliştiricinin local ortamında yaptığı işlem production kullanıcı verilerini değiştirmemelidir.

---

5. Backend Environment Variables

Flask backend environment variables üzerinden yapılandırılmalıdır.

Örnek:

DATABASE_URL=
JWT_SECRET_KEY=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
FLASK_ENV=

İleride:

CORS_ORIGINS=
LOG_LEVEL=
JWT_ACCESS_TOKEN_EXPIRES=

gibi değişkenler eklenebilir.

---

6. .env Kullanımı

Development ortamında:

.env

dosyası kullanılabilir.

Örnek:

DATABASE_URL=postgresql://username:password@localhost:5432/smart_chef_dev
JWT_SECRET_KEY=development-secret
GOOGLE_CLIENT_ID=development-client-id
GOOGLE_CLIENT_SECRET=development-client-secret
FLASK_ENV=development

Ancak:

.env

dosyası Git repository'sine gönderilmemelidir.

`.gitignore` içerisinde:

.env

bulunmalıdır.

---

7. `.env.example`

Repository'de gerçek secret içermeyen:

.env.example

dosyası bulunmalıdır.

Örnek:

DATABASE_URL=
JWT_SECRET_KEY=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
FLASK_ENV=
CORS_ORIGINS=

Bu dosyanın amacı yeni geliştiricinin gerekli environment variables'ları anlamasını sağlamaktır.

Gerçek secret değerleri kesinlikle burada bulunmamalıdır.

---

8. Secret Management

Aşağıdaki bilgiler secret olarak değerlendirilmelidir:

JWT_SECRET_KEY
GOOGLE_CLIENT_SECRET
DATABASE_PASSWORD
Production database credentials
API keys

Bu bilgiler:

- GitHub repository'sine yazılmamalı
- Source code içerisinde hard-code edilmemeli
- Flutter application içerisine secret olarak gömülmemeli
- Loglara yazılmamalıdır.

Production'da platformun secret/environment variable mekanizması kullanılmalıdır.

---

9. Flutter Environment Configuration

Flutter uygulaması da environment'a göre API adresini değiştirebilmelidir.

Örneğin:

Development:
http://10.0.2.2:5000/api

Testing:
https://test-api.example.com/api

Production:
https://api.example.com/api

API URL'i doğrudan widget içerisinde yazılmamalıdır.

Yanlış:

Dio().get("https://api.example.com/api/recipes");

Doğru yaklaşım:

ApiConstants
↓
Environment Configuration
↓
Dio Client

---

10. Flutter API Configuration

Önerilen yapı:

```
lib/
└── core/
    └── constants/
        └── api_constants.dart
```

ve environment'a göre configuration:

Development
Testing
Production

ayrımıdır.

Amaç, production API adresinin uygulamanın farklı noktalarında tekrar tekrar yazılmasını engellemektir.

---

11. Local Development Architecture

Geliştirme sırasında:

Flutter Emulator / Physical Device
↓
Local Flask
↓
Local PostgreSQL

kullanılabilir.

Örnek:

Flutter
↓
http://10.0.2.2:5000
↓
Flask
↓
localhost PostgreSQL

Android Emulator'da host makinenin localhost'una erişim için `10.0.2.2` kullanılması gerekebilir.

Physical device kullanıldığında bilgisayarın local network IP adresi gerekebilir.

---

12. Backend Server

Production Flask uygulaması development server ile çalıştırılmamalıdır.

Yanlış:

flask run

production deployment için kullanılmamalıdır.

Production'da Flask uygulamasının önünde uygun bir WSGI server kullanılmalıdır.

Örneğin:

Gunicorn

kullanılabilir.

Genel yapı:

Internet
↓
HTTPS / Reverse Proxy
↓
Gunicorn
↓
Flask Application

---

13. Application Factory

Backend:

app/__init__.py

içerisinde:

create_app()

application factory yaklaşımı kullanılmalıdır.

Deployment sırasında:

Gunicorn
↓
create_app()
↓
Flask Application

şeklinde application oluşturulabilir.

Bu yaklaşım development ve testing ortamlarının ayrılmasını kolaylaştırır.

---

14. Production Flask Configuration

Production ortamında:

DEBUG = False

olmalıdır.

Ayrıca:

TESTING = False

olmalıdır.

Production hata response'larında:

SQLAlchemy traceback

Python traceback
Database credentials
Internal file paths

gibi bilgiler kullanıcıya gösterilmemelidir.

---

15. PostgreSQL Deployment

Production database:

PostgreSQL

olacaktır.

Database application server'dan ayrı tutulmalıdır.

Önerilen yapı:

Flask Server


PostgreSQL Server

Database'in doğrudan public internetten erişilebilir olması tercih edilmemelidir.

---

16. Database Migration

Database schema değişiklikleri doğrudan production database üzerinde manuel SQL çalıştırılarak yönetilmemelidir.

Proje:

Flask-Migrate
+
Alembic

kullanacaktır.

Örnek migration süreci:

SQLAlchemy Model Change
↓
Migration Generate
↓
Migration Review
↓
Migration Apply

Örneğin:

Add new column

için migration oluşturulmalıdır.

---

17. Migration Safety

Production migration uygulanmadan önce:

1. Migration development ortamında çalıştırılmalıdır.
2. Test database üzerinde çalıştırılmalıdır.
3. Automated tests çalıştırılmalıdır.
4. Migration sonucu kontrol edilmelidir.
5. Production backup alınmalıdır.
6. Migration production'a uygulanmalıdır.

Özellikle destructive migration'lar dikkatle ele alınmalıdır.

Örneğin:

DROP COLUMN
DROP TABLE

işlemleri doğrudan uygulanmamalıdır.

---

18. Database Backup

Production PostgreSQL için düzenli backup alınmalıdır.

Minimum yaklaşım:

Regular Automated Backup
+
Backup Retention

olmalıdır.

Backup stratejisi kullanılan deployment sağlayıcısına göre uygulanacaktır.

Backup yalnızca alınmış olmakla kalmamalı, belirli aralıklarla restore edilerek gerçekten kullanılabilir olduğu kontrol edilmelidir.

---

19. Database Restore Test

Restore süreci:

Backup
↓
Restore Test Database
↓
Database Integrity Check
↓
Application Connection Test

şeklinde test edilmelidir.

Amaç backup dosyasının yalnızca mevcut olduğunu değil, gerektiğinde kullanılabildiğini doğrulamaktır.

---

20. HTTPS

Production API yalnızca HTTPS üzerinden sunulmalıdır.

Doğru:

https://api.example.com

Yanlış:

http://api.example.com

Özellikle authentication JWT kullandığı için HTTP üzerinden token gönderilmemelidir.

Flutter:

Authorization: Bearer <JWT>

header'ını yalnızca güvenli bağlantı üzerinden göndermelidir.

---

21. Reverse Proxy

Production'da aşağıdaki yapı kullanılabilir:

Internet
↓
Reverse Proxy
↓
Gunicorn
↓
Flask

Reverse proxy:

- HTTPS termination
- Request forwarding
- Basic security headers
- Static/network handling
- Connection management

gibi görevlerde kullanılabilir.

Örneğin:

Nginx

kullanılabilir.

---

22. CORS

Backend yalnızca gerekli origin'lere izin vermelidir.

Development:

localhost / local development origins

Production:

defined application origins

kullanılmalıdır.

CORS:

Allow All Origins

şeklinde production'da sınırsız bırakılmamalıdır.

---

23. API Rate Limiting

MVP'nin ilk versiyonunda zorunlu olmasa da authentication ve public recommendation endpoint'leri için rate limiting ilerleyen aşamada eklenmelidir.

Özellikle:

POST /api/auth/social
POST /api/recipes/recommend

gibi endpoint'ler kötüye kullanım açısından takip edilmelidir.

Future:

Rate Limiter
↓
IP / User based limits

uygulanabilir.

---

24. Logging

Backend içerisinde structured logging kullanılmalıdır.

Log seviyeleri:

DEBUG
INFO
WARNING
ERROR
CRITICAL

olarak ayrılabilir.

Production'da gereksiz `DEBUG` logları kapalı tutulmalıdır.

---

25. Log İçeriği

Örnek uygun log:

INFO Recipe recommendation completed
user_id=<internal-id>
recipe_count=8
duration_ms=120

Uygun olmayan log:

JWT_TOKEN=eyJ...
GOOGLE_ID_TOKEN=...
DATABASE_PASSWORD=...

Authentication tokenları ve secret değerler loglanmamalıdır.

---

26. Request ID

API request'lerinin takip edilebilmesi için request ID kullanılabilir.

Örnek:

X-Request-ID: 8b1c...

Backend logları:

request_id
endpoint
HTTP method
status code
duration

gibi bilgileri içerebilir.

Bu özellikle production'daki hataların takip edilmesini kolaylaştırır.

---

27. Monitoring

Production sisteminde temel monitoring bulunmalıdır.

Takip edilmesi gerekenler:

API availability
HTTP 5xx rate
Response time
Database connectivity
Database errors
Authentication failures
Application crashes

İlk MVP'de kapsamlı monitoring sistemi kurulması zorunlu değildir.

Ancak production ortamının tamamen gözlemlenmeden bırakılması önerilmez.

---

28. Health Check

Backend için:

GET /api/health

endpoint'i oluşturulabilir.

Başarılı response:

```
{
  "success": true,
  "data": {
    "status": "ok"
  },
  "message": "Service is healthy."
}
```

Health endpoint'i temel olarak uygulamanın çalıştığını kontrol etmek için kullanılacaktır.

---

29. Database Health

İlerleyen aşamada:

GET /api/health

endpoint'i database bağlantısını da kontrol edebilir.

Örneğin:

Application = OK
Database = OK

veya:

Application = OK
Database = ERROR

durumu monitoring sistemine aktarılabilir.

Ancak hassas database bilgileri response içerisinde gösterilmemelidir.

---

30. CI/CD

GitHub repository üzerinde CI/CD pipeline kurulması önerilmektedir.

Temel akış:

Developer
↓
Git Push
↓
Pull Request
↓
CI
↓
Tests
↓
Static Analysis
↓
Build
↓
Merge
↓
Deployment

---

31. Backend CI Pipeline

Backend için:

Install Python
↓
Install Dependencies
↓
Run Lint
↓
Run Unit Tests
↓
Run Integration Tests
↓
Build / Package

çalıştırılabilir.

Örneğin:

pytest

temel test komutudur.

---

32. Flutter CI Pipeline

Flutter için:

Install Flutter
↓
flutter pub get
↓
flutter analyze
↓
flutter test
↓
Build APK

şeklinde pipeline oluşturulabilir.

Production release aşamasında ayrıca:

Android App Bundle

oluşturulabilir.

---

33. Pull Request Rules

Pull Request açıldığında minimum olarak:

[ ] Backend tests pass
[ ] Flutter tests pass
[ ] Flutter analyze pass
[ ] No obvious secrets
[ ] Migration reviewed
[ ] API contract reviewed

kontrol edilmelidir.

Critical test başarısızsa merge yapılmamalıdır.

---

34. Branch Strategy

Proje için basit bir Git branch yapısı yeterlidir.

Önerilen:

main
develop
feature/*
fix/*

Örneğin:

feature/cart-management
feature/google-auth
feature/recipe-recommendation
fix/favorite-duplicate

şeklinde branch'ler kullanılabilir.

---

35. Main Branch

`main` production'a hazır kodu temsil etmelidir.

Doğrudan:

main

üzerinde geliştirme yapılmaması önerilir.

Değişiklikler Pull Request üzerinden birleştirilmelidir.

---

36. Develop Branch

`develop` geliştirme aşamasındaki entegre kodu temsil edebilir.

Akış:

feature/*
↓
develop

↓
Testing
↓
main

şeklinde olabilir.

Küçük bir öğrenci projesinde branch yapısının gereksiz karmaşık hale getirilmemesi önemlidir.

---

37. Commit Convention

Commit mesajları anlamlı olmalıdır.

Örnek:

feat: add recipe recommendation endpoint
feat: implement cart merge logic
fix: prevent duplicate favorites
test: add recipe matcher tests
refactor: separate cart service
docs: update API specification

Kaçınılması gereken:

update
changes
final
last
new
asdf

gibi belirsiz commit mesajlarıdır.

---

38. Docker Kullanımı

Backend deployment için Docker kullanılabilir.

Önerilen yapı:

backend/
├── Dockerfile
├── .dockerignore
└── requirements.txt

Docker container:

Docker
↓
Gunicorn
↓
Flask

şeklinde çalışabilir.

PostgreSQL production database'in de aynı container içerisinde tutulması önerilmez.

---

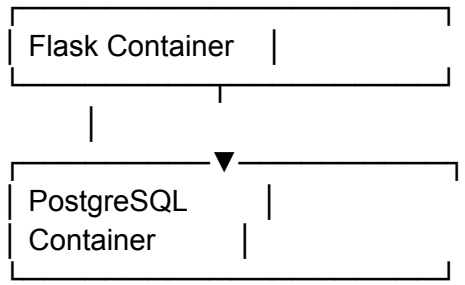
39. Docker Compose — Development

Development ortamında:

docker-compose.yml

veya güncel Docker Compose yaklaşımı kullanılabilir.

Örnek mimari:



Bu yapı geliştiricilerin aynı environment'ı daha kolay kurmasını sağlayabilir.

---

40. Production Containerization

Production deployment platformuna göre Flask:

Docker Container

içerisinde çalıştırılabilir.

Container içerisinde:

Gunicorn
+
Flask Application

bulunur.

Environment variables container dışından inject edilmelidir.

Secret değerleri Docker image içerisine yazılmamalıdır.

---

41. Docker Image Güvenliği

Docker image:

- Gereksiz paketler içermemeli
- Secret içermemeli
- `.env` dosyasını image'a kopyalamamalı
- Güncel base image kullanmalı
- Mümkünse non-root user ile çalışmalıdır

Örneğin:

COPY .env .

gibi bir işlem production Dockerfile içerisinde bulunmamalıdır.

---

42. Flutter Release Build

Android için production build:

flutter build appbundle --release

ile oluşturulabilir.

Test APK'sı:

flutter build apk --release

ile oluşturulabilir.

Production build'de:

Production API URL
Production configuration
Release signing

kullanılmalıdır.

---

43. Android Signing

Production Android uygulaması release signing ile imzalanmalıdır.

Keystore bilgileri:

Git repository

içerisinde tutulmamalıdır.

Signing credentials güvenli şekilde saklanmalıdır.

---

44. iOS Release

İleride iOS desteği eklenirse:

Flutter
↓
iOS Build
↓
Code Signing

↓
App Store Connect

akışı kullanılabilir.

Apple signing certificates ve provisioning bilgileri güvenli şekilde saklanmalıdır.

---

45. Database Migration During Deployment

Production deployment sırası dikkatli belirlenmelidir.

Önerilen:

1. Backup
2. Deploy compatible backend
3. Run migration
4. Verify database
5. Health check
6. Run smoke tests
7. Complete deployment

Migration ile yeni backend kodunun birbirine uyumlu olması önemlidir.

---

46. Backward Compatibility

API değişikliklerinde Flutter uygulamasının eski sürümlerinin tamamen kırılmaması mümkün olduğunca gözetilmelidir.

Örneğin bir alan kaldırılacaksa:

Old API
↓
New API

geçişi kontrollü yapılmalıdır.

Breaking API changes gerektiğinde API versioning düşünülebilir.

Örneğin:

/api/v1
/api/v2

MVP'de `/api` yapısı yeterlidir.

---

47. Deployment Smoke Tests

Deployment sonrasında minimum smoke testler çalıştırılmalıdır:

GET /api/health

GET /api/ingredients

POST /api/recipes/recommend

Authentication:

POST /api/auth/social

için uygun test environment kontrolü yapılmalıdır.

Production'da gerçek kullanıcı authentication bilgileriyle otomatik test yapılmamalıdır.

---

48. Rollback Strategy

Deployment başarısız olursa önceki çalışan versiyona dönülebilmelidir.

Genel akış:

New Release
↓
Health Check
↓
Failure
↓
Rollback
↓

Previous Release

Database migration rollback her zaman otomatik ve risksiz kabul edilmemelidir.

Bu nedenle destructive migration'lar öncesinde backup alınması zorunlu tutulmalıdır.

---

49. Incident Response

Production'da kritik hata oluştuğunda:

Detect
↓
Identify
↓
Contain
↓
Fix / Rollback
↓
Verify
↓
Document

akışı uygulanmalıdır.

Örneğin API tamamen çalışmıyorsa önce:

Health check
Application logs
Database connectivity
Recent deployment

kontrol edilmelidir.

---

50. Production Security Checklist

Production öncesinde:

[ ] DEBUG disabled
[ ] HTTPS enabled

[ ] Secrets outside source code
[ ] Production database isolated
[ ] JWT secret configured
[ ] Google credentials configured
[ ] CORS restricted
[ ] Database credentials protected
[ ] Sensitive logs disabled
[ ] Error details hidden
[ ] Admin authorization active
[ ] User ownership checks active
[ ] Backup configured

olmalıdır.

---

51. Development Checklist

Yeni geliştirici projeyi çalıştırmadan önce:

[ ] Repository cloned
[ ] Flutter installed
[ ] Python installed
[ ] PostgreSQL available
[ ] .env created
[ ] Dependencies installed
[ ] Database created
[ ] Migrations applied
[ ] Backend started
[ ] Flutter app started
[ ] API connection tested

kontrol edilmelidir.

---

52. Backend Local Setup

Genel geliştirme akışı:

Create virtual environment
↓
Install requirements
↓

Configure .env
↓
Create PostgreSQL database
↓
Run migrations
↓
Start Flask/Gunicorn
↓
Test /api/health

Örnek dependency installation:

pip install -r requirements.txt

Migration:

flask db upgrade

Development server:

flask run

Production:

Gunicorn

kullanılmalıdır.

---

53. Flutter Local Setup

Genel akış:

Flutter SDK
↓
flutter pub get
↓
Configure API URL
↓
Connect Emulator / Device
↓
flutter run

Kod kalitesi kontrolü:

flutter analyze

Test:

flutter test

---

54. Release Versioning

Uygulama sürümleri takip edilmelidir.

Örnek:

1.0.0
1.0.1
1.1.0
2.0.0

Genel yaklaşım:

Major.Minor.Patch

olarak tutulabilir.

Örneğin:

1.0.0

ilk MVP release.

Bug fix:

1.0.1

Yeni backward-compatible feature:

1.1.0

Breaking change:

2.0.0

olarak değerlendirilebilir.

---

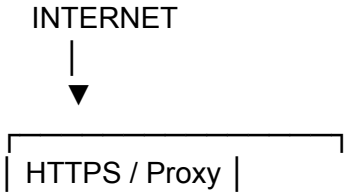
55. MVP Deployment Plan

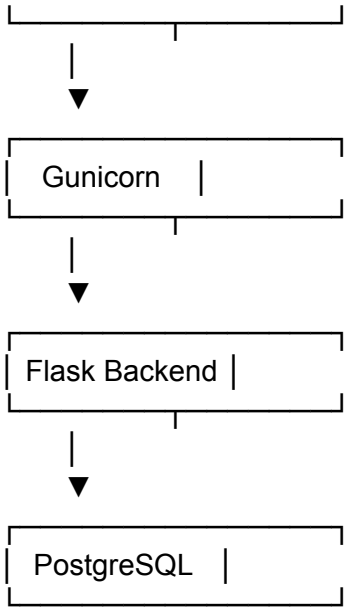
MVP için önerilen deployment sırası:

1. Local Development
   ↓
2. Backend Unit Tests
   ↓
3. Backend Integration Tests
   ↓
4. Flutter Tests
   ↓
5. Test Environment
   ↓
6. API Smoke Tests
   ↓
7. Flutter Release Build
   ↓
8. Production Backend
   ↓
9. Production Database Migration
   ↓
10. Health Check
   ↓
11. Production Mobile Build
   ↓
12. Final Smoke Test

---

56. MVP Deployment Mimarisi





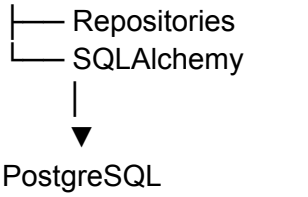
Flutter Application
HTTPS ► Flask API

---

57. Deployment ile Uygulama Mimarisi İlişkisi

Final sistem:

Flutter
├── Auth
├── Ingredients
├── Recipes
├── Favorites
├── Cart
└── Profile
HTTPS REST API
Flask
├── Routes
├── Schemas
├── Services
├── AI / Matching



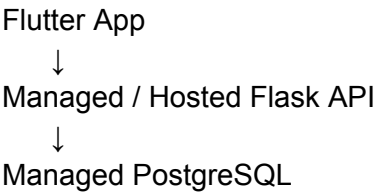
Bu yapı önceki dokümanlarda tanımlanan Clean Architecture ve backend layered architecture ile uyumludur.

---

58. MVP'de Kullanılacak Deployment Yaklaşımı

MVP aşamasında gereksiz infrastructure karmaşıklığından kaçınılmalıdır.

Başlangıçta:



yeterlidir.

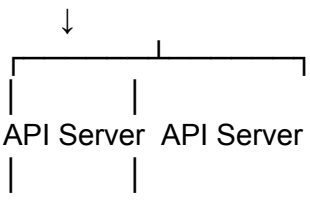
İlk sürüm için Kubernetes, microservices, service mesh veya karmaşık distributed architecture zorunlu değildir.

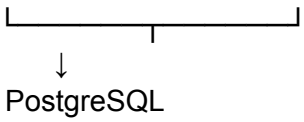
Projenin kullanıcı sayısı ve trafik ihtiyacı arttığında infrastructure genişletilebilir.

---

59. Gelecekte Ölçeklendirme

Kullanıcı ve trafik artarsa:





şeklinde horizontal scaling uygulanabilir.

Ayrıca ilerleyen sürümlerde:

Redis
Background Workers
Caching
CDN
Object Storage

gibi bileşenler eklenebilir.

Ancak bunlar MVP kapsamında değildir.

---

60. Future CI/CD Improvements

İlerleyen sürümlerde pipeline:

Pull Request
↓
Lint
↓
Unit Test
↓
Integration Test
↓
Security Scan
↓
Build
↓
Staging Deployment
↓
Smoke Test
↓
Production Deployment

haline getirilebilir.

Production deployment için manuel approval da eklenebilir.

---

61. Deployment Test Matrix

Alan	Development	Testing	Production
Debug	Açık olabilir	Kapalı	Kapalı
Test DB	Hayır	Evet	Hayır
Production DB	Hayır	Hayır	Evet
Test secrets	Evet	Evet	Hayır
Production secrets	Hayır	Hayır	Evet
HTTPS	Önerilir	Evet	Zorunlu
Automated tests	Evet	Evet	Dolaylı
Backup	Gerekmeyebilir	Gerekmeyebilir	Zorunlu
Monitoring	Temel	Temel	Evet

---

62. Definition of Deployment Done

Bir deployment tamamlanmış kabul edilmeden önce:

[ ] Correct environment selected
[ ] Environment variables configured
[ ] Secrets configured
[ ] Database reachable
[ ] Migration completed
[ ] Backend started
[ ] Health endpoint works
[ ] API smoke tests pass
[ ] Authentication works
[ ] Recommendation works
[ ] Favorites works
[ ] Cart works
[ ] Flutter connects to correct API
[ ] Production debug disabled
[ ] HTTPS active
[ ] Logs checked

[ ] Backup verified

kontrol edilmelidir.

---

63. Final Deployment Rules

Smart Chef & Pantry AI için temel deployment kuralları:

1. Development, Testing ve Production ortamları ayrılacaktır.
2. Production database development ortamından kullanılmayacaktır.
3. Secret bilgiler source code içerisinde tutulmayacaktır.
4. Production Flask development server ile çalıştırılmayacaktır.
5. Production API HTTPS üzerinden sunulacaktır.
6. Database migration Flask-Migrate/Alembic ile yönetilecektir.
7. Production migration öncesinde backup alınacaktır.
8. Authentication tokenları loglanmayacaktır.
9. Production'da debug bilgileri kullanıcıya gösterilmeyecektir.
10. Kritik testler CI/CD pipeline içerisinde çalıştırılacaktır.
11. Main branch production-ready kodu temsil edecektir.
12. Deployment sonrasında health check ve smoke test yapılacaktır.
13. Kritik deployment hatalarında rollback prosedürü uygulanacaktır.
14. Production database düzenli olarak yedeklenecektir.
15. MVP'de infrastructure gereksiz şekilde karmaşıklaştırılmayacaktır.

---

64. Sonuç

Smart Chef & Pantry AI'nin deployment yapısı, uygulamanın geliştirme aşamasından production ortamına güvenli ve kontrollü şekilde taşınmasını sağlayacak şekilde tasarlanmıştır.

Temel yaklaşım:

Development
↓
Testing
↓
CI/CD
↓
Staging / Verification
↓
Production
↓
Monitoring
↓
Backup / Rollback

şeklindedir.

MVP aşamasında temel hedef; güvenli bir Flask API, yönetilebilir PostgreSQL database ve production'a uygun Flutter release build oluşturmaktır.

Sistem büyüdükçe caching, load balancing, background workers, CDN, advanced monitoring ve horizontal scaling gibi bileşenler eklenebilir.

Bu doküman, projenin kodlama aşamasından deployment aşamasına kadar environment ve operasyonel standartlarını belirleyen temel referans doküman olarak kullanılacaktır.